

Software Engineering Project
Academic Year 2026–2027

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

SmartExpense: Smart Expense Tracker & Event-Based Savings Planner

*A Browser-Based Personal Finance Dashboard with
Event Savings Goals and Linked Checklists*

Submitted by:

[1024030441] [Divanshi garg]

[1024030438] [Navya Arora]

B.E. Third Year, Computer Science & Engineering

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Frontend Prototype Evaluation Report

Computer Science and Engineering Department

Contents

Chapter 1: Introduction	3
1.1 Background and Context.....	3
1.1.1 Problem Statement.....	3
1.2 Project Objectives	3
Chapter 2: Methodology and Design.....	5
2.1 System Architecture.....	5
2.1.1 High-Level Design	5
2.1.2 Technical Stack	5
2.2 Implementation Details	6
2.2.1 Module 1: Core Screens	6
2.2.2 Module 2: Secondary Features	7
Chapter 3: Results and Evaluation.....	8
3.1 Testing Methodology	8
3.2 Feature Completion Status	8
3.3 Analysis.....	9
Chapter 4: Conclusion.....	10
4.1 Summary of Work.....	10
4.2 Key Findings	10
4.3 Future Work and Recommendations.....	10
4.4 Final Remarks	10

Chapter 1: Introduction

1.1 Background and Context

Keeping track of everyday spending is a problem most people solve badly: expenses are scattered across cash, UPI, cards and bill payments, and by the time a monthly statement arrives it is too late to change anything. This becomes noticeably harder around specific occasions a festival, a wedding, a birthday, a trip where a person is not just spending day-to-day but is also trying to save towards a fixed, dated goal alongside their regular budget. Most existing expense trackers either present a generic monthly ledger with no link to upcoming events, or they focus purely on categorisation without helping the user plan a checklist of what still needs to be paid for.

SmartExpense is proposed as a personal finance web application that closes this gap. It gives a user a single dashboard for monthly spending by category, a running list of transactions (added manually or imported from a CSV export), a set of savings events with a target budget and progress bar, and checklists of tasks/items linked to each event, so that day-to-day spending and event-based saving are visible side by side rather than tracked in separate tools.

1.1.1 Problem Statement

Individuals lack a single, lightweight tool that connects ordinary monthly expense tracking with saving towards specific upcoming events. Spreadsheets and generic budgeting apps typically treat all spending as one undifferentiated stream, so a user cannot easily answer "how much have I actually saved towards Diwali" or "what is left to arrange for the wedding" without maintaining a second, disconnected list. SmartExpense addresses this by pairing a category-wise spending dashboard with per-event savings tracking and linked checklists inside one interface, so a user's regular budget and their event-specific goals are always visible together.

1.2 Project Objectives

The core objectives of the SmartExpense project comprise:

- **Centralised Spending Dashboard:** Give the user a single monthly overview of total spend, remaining budget, upcoming events and a category-wise spending breakdown.
- **Transaction Logging:** Allow transactions to be added manually or bulk-imported from a CSV file, each tagged with a merchant, category, amount, date and entry type.
- **Event-Based Savings Tracking:** Let the user create events with a target date and budget, and track amount saved against that budget with a visual progress indicator and urgency badge based on days remaining.
- **Linked Checklists:** Offer per-event notes/checklists (e.g. a Diwali shopping list or a wedding checklist) so planning tasks live next to the event they belong to.

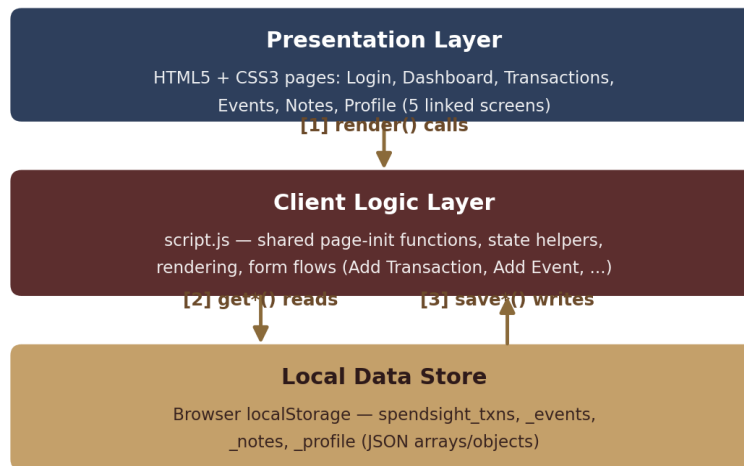
- Consistent, Accessible Interface: Provide a single sidebar-navigated layout (Dashboard, Transactions, Events, Notes, Profile) shared across all screens for a predictable user experience.
- Foundation for a Full-Stack Build: Establish a complete, working frontend prototype — validated against realistic sample data as the basis for the backend, database and (later) predictive-spending service planned for the next project phase.

Chapter 2: Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

At the current, frontend-only stage, SmartExpense is organised into three layers running entirely in the browser: a Presentation Layer of static HTML/CSS pages, a Client Logic Layer (script.js) that reads and writes application state, and a Local Data Store held in the browser's localStorage. Figure 2.1 illustrates this arrangement, along with the Application Layer that is planned but not yet built (Section 4.3).



Planned (not yet implemented): Application Layer (REST API) + persistent database — see Section 4.3

Figure 2.1: SmartExpense Frontend-Prototype Architecture (backend layer planned)

The working flow today is: a page loads to script.js seeds sample data on first run only then the relevant get*() helper reads the matching localStorage key (transactions, events, notes or profile) then the page's own render function builds the on-screen cards, tables or grids from that data. Any add/edit action writes back through a matching save*() helper and immediately re-renders, so the interface always reflects localStorage. No network request is made anywhere in the current build.

2.1.2 Technical Stack

- **Markup & Styling:** HTML5 and hand-written CSS3 (styles.css for the app shell, a separate style.css for the login screen), using CSS variables for a shared colour palette and backdrop-filter "glass" panels.
- **Typography:** Google Fonts — Playfair Display for headings/branding, Inter for body text and UI labels.
- **Client Scripting:** Vanilla JavaScript (script.js), with no framework or build step — a single shared file whose functions are conditionally invoked per page based on which DOM elements are present.

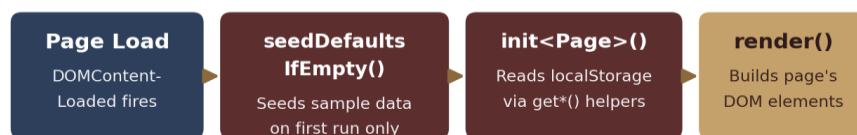
- Client-Side Persistence: Browser localStorage, used as the only data store at this stage (keys: spendsight_txns, spendsight_events, spendsight_notes, spendsight_profile).
- Icons: Font Awesome (login page) and inline Unicode glyphs (sidebar navigation icons).
- Development Environment: Standard HTML/CSS/JS editing, tested directly in the browser; no backend runtime is required to run the current prototype.
- Version Control: Git, for tracking changes as the prototype evolves.

2.2 Implementation Details

2.2.1 Module 1: Core Screens

The core module implements the shared page shell and the Dashboard, which together establish the data model every other screen depends on. Figure 2.2 shows the render pipeline that runs on every page load.

- Login screen (index.html) — email/password form with client-side required-field validation; on submit, redirects straight to the Dashboard (no authentication service yet — see Section 4.3).
- Shared sidebar/shell — present on all five app screens, showing the SmartExpense logo, five navigation links, the active-page highlight, and a logged-in user name/email pulled from the stored profile.
- Dashboard (dashboard.html) — four summary cards (spent this month, budget remaining, upcoming events, a placeholder "Predicted" card reserved for a future ML service), a category-wise spending bar chart, and "recent transactions" / "upcoming events" panels, each with its own empty state.
- seedDefaultsIfEmpty() — populates localStorage with a realistic sample profile, 10 transactions, 5 events and 5 linked note sets on first run, which was used to validate every screen end-to-end without needing a backend.



Runs identically on every page load — each page only wires up the blocks its own elements need

Figure 2.2: Core Page-Load and Render Pipeline

2.2.2 Module 2: Secondary Features

Building on the core shell, three further screens extend SmartExpense to its full proposed scope of tracking, saving and planning. Figure 2.3 shows how each screen reads and writes the same shared local data store.

- Transactions (transactions.html) — a sortable table of all transactions with merchant, category, amount, date and entry type; supports adding a single transaction via prompts and bulk-adding rows from an uploaded CSV file.
- Events (events.html) — a card grid of savings events, each showing the target date, a days-remaining badge (urgent / soon / ok based on proximity), the budget, and a progress bar for amount saved versus budget.
- Notes (notes.html) — a card grid of checklists, each tagged to a specific event, with checkable items whose done state is persisted back to localStorage immediately on click.
- Profile (profile.html) — a form for name, email and monthly budget (₹) that writes to the shared profile record and updates the sidebar user display across the app.

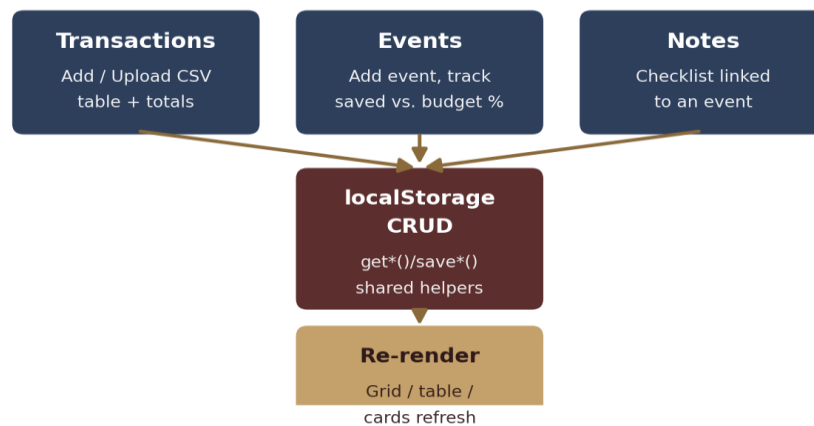


Figure 2.3: Secondary Feature Flow — Transactions, Events and Notes Sharing One Data Store

Chapter 3: Results and Evaluation

3.1 Testing Methodology

Since the prototype has no backend yet, testing was carried out entirely in the browser: each of the six screens (Login, Dashboard, Transactions, Events, Notes, Profile) was loaded in sequence and exercised manually, with the browser's DevTools console and DOM inspector used to confirm that elements populated correctly and that no script errors occurred. The Application tab's localStorage view was used to inspect the underlying JSON after every add/edit action, confirming that SmartExpense_txns, SmartExpense_events, SmartExpense_notes and SmartExpense_profile were being written in the expected shape. Figure 3.1 summarises this framework.



Figure 3.1: Manual, Page-by-Page Testing Framework

This process also surfaced and resolved a data-consistency issue during development: the login page's redirect and the sidebar's "log out" link both point to static HTML pages with no session concept, so a user can currently reach the Dashboard directly by URL without "logging in" at all. This is expected at the frontend-only stage and is scheduled to be closed once real authentication is added (Section 4.3).

3.2 Feature Completion Status

Formal performance testing (API response time, database load, prediction accuracy) is not yet applicable, since no backend, database or ML service exists yet in this prototype. Table 3.1 instead summarises the planned end-to-end capabilities and their current status.

Capability	Current State	Status
Dashboard summary cards & category chart	Working, from localStorage	Done (frontend)
Add transaction / CSV import	Working, prompt-based UI	Done (frontend)
Event creation & savings progress	Working, prompt-based UI	Done (frontend)
Event-linked checklists (Notes)	Working, toggle persists	Done (frontend)
Profile & budget editing	Working, writes to profile	Done (frontend)

Capability	Current State	Status
User authentication / sessions	Form only, no backend check	Pending
Persistent, multi-device storage	Single-browser localStorage only	Pending
"Predicted — Oct" spend forecast	Placeholder card in UI	Pending
Spending-anomaly alert banner	Markup present, unwired	Pending

Table 3.1: SmartExpense Feature Completion Status at Frontend-Prototype Stage

3.3 Analysis

At the current stage, the frontend prototype reliably demonstrates the full intended user experience dashboard overview, transaction logging, event-based saving and linked checklists using seeded and user-entered sample data held in the browser. This confirms that the data model (transactions, events, notes and profile as four related JSON collections) and the page-to-page navigation flow are sound and cover the objectives set out in Section 1.2. The main remaining work is replacing localStorage with a real backend and database, adding authentication, and wiring the two placeholder UI elements (the predicted-spend card and the anomaly alert) to an actual analysis service, as described in Section 4.3.

Chapter 4: Conclusion

4.1 Summary of Work

This project set out to build SmartExpense, a web application that lets a user track everyday spending and save towards specific upcoming events from one dashboard. A frontend-only prototype was designed and implemented across five linked screens Dashboard, Transactions, Events, Notes and Profile plus a login screen, sharing one client-side data model held in browser localStorage. The prototype was validated by seeding realistic sample data and manually exercising every screen and every add/edit interaction.

4.2 Key Findings

- A four-collection client-side model : transactions, events, notes and profile is sufficient to represent the full SmartExpense user experience and supports the dashboard, table, card-grid and checklist views the app needs.
- Keeping all page logic in one shared script.js file, gated by checks for each page's own key elements, made it straightforward to reuse helpers (formatting, date math, localStorage get/save) across all six screens without duplication.
- Building against localStorage first was an effective way to validate the full UI and interaction flow end-to-end before committing to a backend schema, and surfaced the missing-session gap described in Section 3.1 early.

4.3 Future Work and Recommendations

For the next project phase, the team will implement:

- A backend REST API and persistent database to replace localStorage, so data survives across devices and browser sessions.
- Real user authentication (sign-up, login sessions, logout) so the Login screen and the sidebar "Log out" link are backed by an actual session rather than a static redirect.
- A predictive-spending service to power the Dashboard's "Predicted Oct" card, and rule-based or model-based logic to drive the currently-unwired spending-anomaly alert.
- Server-side validation of transactions and CSV imports, replacing the current prompt-based, client-only input flow.
- Migration of the Add Transaction / Add Event / Add Note flows from browser prompt() dialogs to proper in-page forms and modals.
- Carry out load, response-time and usability evaluation once the system is backend-complete.

4.4 Final Remarks

SmartExpense addresses a genuine, everyday problem spending and event-based saving being tracked in separate, disconnected tools. The prototype work completed so far demonstrates that the core idea is workable as a single, coherent interface: a shared data model can drive a dashboard, a transaction ledger, savings-goal tracking and linked checklists from one lightweight client-side codebase. The remaining work is primarily one of moving that proven interface onto a real backend and validating it at realistic scale.